LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 5 – RELASI ANTAR KELAS

Khairun Fahmi

103132400035

S1SD-05-A

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Diagram kelas adalah diagram yang digunakan untuk memodelkan kelas-kelas dalam sebuah sistem beserta hubungan antar kelasnya. Setiap kelas digambarkan dengan kotak yang terdiri dari nama kelas, atribut, dan method, dengan simbol access modifier berupa tanda + untuk public, - untuk private, dan # untuk protected. Hubungan antar kelas digambarkan dengan garis, dan modul ini membahas empat jenis hubungan, yaitu asosiasi, agregasi, komposisi, serta generalisasi dan spesialisasi yang terkait dengan pewarisan. Asosiasi adalah hubungan antara dua kelas terpisah yang terbentuk melalui objek pada tiap kelas, misalnya kelas Dosen yang memanggil method milik kelas Mahasiswa melalui parameter pada method giveScore() dan getScore(), tanpa menjadikan objek Mahasiswa sebagai atribut. Agregasi adalah hubungan yang lebih ketat antara dua kelas dengan konsep bagian (part) dan keseluruhan (whole), namun kedua kelas tetap dapat diciptakan sendiri-sendiri, misalnya kelas Jurusan yang memiliki atribut array bertipe Mahasiswa. Komposisi merupakan bentuk khusus dari agregasi dengan ikatan yang lebih kuat, karena bagian tidak akan ada tanpa kelas yang menjadi keseluruhannya, sehingga ketika kelas Komputer musnah maka CPU, Monitor, dan Mouse ikut musnah dan baru diciptakan saat objek Komputer diinstansiasi. Sementara itu, generalisasi menggambarkan hubungan di mana suatu kelas merupakan turunan dari kelas lain.

1. Asosiasi

Mahasiswa.java

```
package com.mycompany.guidedmodul05;

public class Mahasiswa {
    private String nim;
    private String nama;
    private int nilai;

    public void setNama(String nama) { this.nama = nama; }
    public void setNim(String nim) { this.nim = nim; }
    public String getNama() { return this.nama; }
    public String getNim() { return this.nim; }
    public void setNilai(int nilai) { this.nilai = nilai; }
    public int getNilai() { return this.nilai; }
}
```

Dosen.java

```
 package com.mycompany.guidedmodul05;

public class Dosen {
    private String kodeDosen;
    private String namaDosen;

    public void setKodeDosen(String kodeDosen) { this.kodeDosen =
kodeDosen; }
```

```
    public void setNamaDosen(String namaDosen) { this.namaDosen =
namaDosen; }
    public String getKodeDosen() { return this.kodeDosen; }
    public String getNamaDosen() { return this.namaDosen; }

    public void giveScore(Mahasiswa s, int nilai) {
        s.setNilai(nilai);
    }

    public int getScore(Mahasiswa s) {
        return s.getNilai();
    }

    public static void main(String[] args) {
        Mahasiswa m = new Mahasiswa();
        m.setNim("130118383");
        m.setNama("Budi");

        Dosen d = new Dosen();
        d.giveScore(m, 90);

        System.out.println("Nim: " + m.getNim());
        System.out.println("Nama: " + m.getNama());
        System.out.println("Nilai: " + d.getScore(m));
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ GuidedModul05 ---
Nim: 130118383
Nama: Budi
Nilai: 90
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

2. Agregasi

Mahasiswa.java

```
package guidedmodul05Agregasi;

public class Mahasiswa {
    private String NIM, Nama;
    public Mahasiswa(String no, String nm) {
        this.NIM = no;
        this.Nama = nm;
    }
    public String GetNIM() { return (NIM); }
    public String GetNama() { return (Nama); }
```

```
}
```

Jurusan.java

```
package guidedmodul05Agregasi;

public class Jurusan {
   private String KodeJurusan, NamaJurusan;
   private Mahasiswa[] Daftar = new Mahasiswa[10];
   private int JmlMhs = 0;

   public Jurusan(String kode, String nama) {
      this.KodeJurusan = kode;
      this.NamaJurusan = nama;
   }
   public void AddMahasiswa(Mahasiswa m) {
      this.Daftar[JmlMhs] = m;
      this.JmlMhs++;
   }
   public void DisplayMahasiswa() {
      int i;
      System.out.println("Kode Jurusan : " + this.KodeJurusan);
      System.out.println("Nama Jurusan: " + this.NamaJurusan);
      System.out.println("Daftar Mahasiswa :");
      for(i = 0; i < JmlMhs; i++) {
         System.out.println(Daftar[i].GetNIM() + " " +
Daftar[i].GetNama());
      }
   }

   // Tambahan method main agar bisa di-run (opsional jika diminta)
   public static void main(String[] args) {
      Jurusan j = new Jurusan("IF", "Informatika");
      Mahasiswa m1 = new Mahasiswa("123", "Budi");
      Mahasiswa m2 = new Mahasiswa("124", "Andi");

      j.AddMahasiswa(m1);
      j.AddMahasiswa(m2);
      j.DisplayMahasiswa();
   }
}
```

Output :

```
Kode Jurusan : IF
Nama Jurusan: Informatika
Daftar Mahasiswa :
123 Budi
124 Andi
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

B. Unguided

1. Soal Unguided

Sebuah cafe membutuhkan sistem sederhana untuk mencatat dan memproses pesanan pelanggan. Dalam sistem ini, relasi antar kelas dirancang untuk mencerminkan kondisi nyata operasional cafe.

Jawaban :

ItemPesanan.java

```
package com.mycompany.cafeapp;

public class ItemPesanan {
    private String namaItem;
    private RincianHarga rincian;

    public ItemPesanan(String namaItem, double hargaDasar) {
        this.namaItem = namaItem;
        // Komposisi: RincianHarga dibuat langsung di dalam constructor
        this.rincian = new RincianHarga(hargaDasar);
    }

    public String getNamaItem() {
        return namaItem;
    }

    public RincianHarga getRincian() {
        return rincian;
```

```
    }
}
```

Kasir.java

```
package com.mycompany.cafeapp;

public class Kasir {
    private String namaKasir;

    public Kasir(String namaKasir) {
        this.namaKasir = namaKasir;
    }

    public String getNamaKasir() {
        return namaKasir;
    }

    // Asosiasi: Pesanan hanya diterima lewat parameter, tidak disimpan sebagai
field
    public void prosesPembayaran(Pesanan pesanan) {
        double totalSubtotal = 0;
        double totalPajak = 0;
        ItemPesanan[] items = pesanan.getDaftarItem();

        for (int i = 0; i < pesanan.getJumlahItem(); i++) {
            RincianHarga r = items[i].getRincian();
            totalSubtotal += r.getHargaDasar();
            totalPajak += r.getPajak();
        }

        double totalBayar = totalSubtotal + totalPajak;

        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + namaKasir);
        System.out.println("Pesanan        : Meja " + pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : " + pesanan.getJumlahItem());
        System.out.println("Total Subtotal : Rp " + totalSubtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp " + totalBayar);
        System.out.println("==============================");
    }
}
```

Main.java

```
package com.mycompany.cafeapp;

public class Kasir {
    private String namaKasir;

    public Kasir(String namaKasir) {
```

```
        this.namaKasir = namaKasir;
    }

    public String getNamaKasir() {
        return namaKasir;
    }

    // Asosiasi: Pesanan hanya diterima lewat parameter, tidak disimpan sebagai
field
    public void prosesPembayaran(Pesanan pesanan) {
        double totalSubtotal = 0;
        double totalPajak = 0;
        ItemPesanan[] items = pesanan.getDaftarItem();

        for (int i = 0; i < pesanan.getJumlahItem(); i++) {
            RincianHarga r = items[i].getRincian();
            totalSubtotal += r.getHargaDasar();
            totalPajak += r.getPajak();
        }

        double totalBayar = totalSubtotal + totalPajak;

        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + namaKasir);
        System.out.println("Pesanan        : Meja " + pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : " + pesanan.getJumlahItem());
        System.out.println("Total Subtotal : Rp " + totalSubtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp " + totalBayar);
        System.out.println("==============================");
    }
}
```

Pesanan.java

```
package com.mycompany.cafeapp;

public class Pesanan {
    private int nomorMeja;
    private ItemPesanan[] daftarItem;
    private int jumlahItem;

    public Pesanan(int nomorMeja, int kapasitas) {
        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[kapasitas];
        this.jumlahItem = 0;
    }

    public void tambahItem(ItemPesanan item) {
        if (jumlahItem < daftarItem.length) {
            daftarItem[jumlahItem] = item;
```

```
            jumlahItem++;
        } else {
            System.out.println("Pesanan penuh, item " + item.getNamaItem() + " tidak
    dapat ditambahkan.");
        }
    }

    public void tampilkanPesanan() {
        System.out.println("=== DAFTAR PESANAN CAFE ===");
        System.out.println("Nomor Meja: " + nomorMeja);
        for (int i = 0; i < jumlahItem; i++) {
            ItemPesanan item = daftarItem[i];
            RincianHarga r = item.getRincian();
            System.out.println((i + 1) + ". " + item.getNamaItem()
                    + " - Rp " + r.getHargaTotal()
                    + " (Harga: " + r.getHargaDasar()
                    + ", Pajak: " + r.getPajak() + ")");
        }
    }

    public int getNomorMeja() {
        return nomorMeja;
    }

    public ItemPesanan[] getDaftarItem() {
        return daftarItem;
    }

    public int getJumlahItem() {
        return jumlahItem;
    }
}
```

RincianHarga.java

```
package com.mycompany.cafeapp;

public class RincianHarga {
    private double hargaDasar;
    private double pajak;

    public RincianHarga(double hargaDasar) {
        this.hargaDasar = hargaDasar;
        this.pajak = hargaDasar * 0.10;
    }

    public double getHargaDasar() {
        return hargaDasar;
    }

    public double getPajak() {
        return pajak;
```

```
    }

    public double getHargaTotal() {
        return hargaDasar + pajak;
    }
}
```

Output :

```
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)
=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
==============================
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

Penjelasan :

Program ini terdiri dari lima class yang menunjukkan tiga jenis relasi antar kelas. RincianHarga menyimpan harga dasar dan menghitung pajak 10% secara otomatis di constructor-nya, lalu ItemPesanan membuat objek RincianHarga langsung di dalam constructor-nya dengan new, sehingga keduanya terikat siklus hidupnya (komposisi). Pesanan menampung banyak ItemPesanan dalam array standar ItemPesanan[] dengan penanda jumlahItem, dan item-itemnya dibuat terpisah di Main lalu dimasukkan lewat tambahItem(), sehingga item tetap bisa berdiri sendiri tanpa Pesanan (agregasi). Kasir tidak menyimpan Pesanan sebagai field, melainkan hanya menerimanya sebagai parameter pada prosesPembayaran() untuk menjumlahkan subtotal dan pajak seluruh item lalu mencetak struk (asosiasi). Seluruh field dibuat private dan diakses lewat getter sebagai penerapan enkapsulasi, sedangkan Main bertugas merangkai semua objek dan menguji alurnya hingga menghasilkan total bayar Rp 71500.0 untuk Meja 4.

C. Kesimpulan

Praktikum ini menunjukkan bahwa kelas-kelas dalam pemrograman berorientasi objek dapat saling berhubungan sesuai dengan kondisi sistem yang dimodelkan, dan hubungan tersebut

digambarkan melalui diagram kelas. Pada asosiasi, kelas Dosen hanya menggunakan objek Mahasiswa melalui parameter method tanpa menyimpannya sebagai atribut. Pada agregasi, kelas Jurusan menampung objek Mahasiswa dalam array sebagai bagian darinya, tetapi keduanya tetap dapat dibuat secara terpisah. Pada komposisi, objek CPU, Monitor, dan Mouse merupakan bagian dari Komputer yang tidak dapat berdiri sendiri dan ikut musnah bersama induknya. Dengan demikian, perbedaan ketiga relasi terletak pada kuat lemahnya ikatan antar kelas, yaitu asosiasi paling ringan, agregasi lebih ketat, dan komposisi paling kuat, sehingga pemilihan relasi yang tepat membantu menghasilkan rancangan program yang sesuai dengan kondisi sebenarnya.